MIPS 编程

1： 寄存器种类；

2： 算术及寻址指令

3： 程序结构

4： 系统调用

所有 MIPS 指令都是 32 位长的

1 字节=8 位，半字长=2 个字节，1 字长=4 个字节

一个字符空间=1 个字节

一个整型=一个字长=4 个字节

单个字符用单引号，例如：'b'

字符串用双引号，例如："A string"

MIPS 下一共有 32 个通用寄存器

在汇编中，寄存器标志由$符开头

寄存器表示可以有两种方式

直接使用该寄存器对应的编号，例如：从$0 到$31

使用对应的寄存器名称，例如：$t1, $sp

栈的走向是从高地址到低地址

详细含义：下表格对于乘法和除法分别有对应的两个寄存器$lo, $hi 对于二者，不存在直接寻址；必须要通过 mfhi("move from hi")以及 mflo("move from lo")分别来进行访问对应的内容

Register N0. 寄存器号	Name 寄存器名	Description 寄存器用途
0	zero	the value 0 永远返回零
1	$at	(assembler temporary) reserved by the assembler 汇编保留寄存器（不可做其他用途）
2-3	$v0 - $v1	(values) from expression evaluation and function results （Value 简写）存储表达式或者是函数的返回值
4-7	$a0 - $a3	(arguments) First four parameters for subroutine. Not preserved across procedure calls （Argument 简写）存储子程序的前 4 个参数，在子程序调用过程中释放
8-15	$t0 - $t7	(temporaries) Caller saved if needed. Subroutines can use w/out saving. Not preserved across procedure calls （Temp 简写）临时变量，同上调用时不保存
16-23	$s0 - $s7	(saved values) - Callee saved. A subroutine using one of these must save original and restore it before exiting. Preserved across procedure calls 函数调用时必须保存，调用完成后需要恢复
24-25	$t8 - $t9	(temporaries) Caller saved if needed. Subroutines can use w/out saving. These are in addition to $t0 - $t7 above. Not preserved across procedure calls. （Temp 简写）算是前面$0~$7 的一个继续，属性同$t0~$t7
26-27	$k0 - $k1	reserved for use by the interrupt/trap handler （kernel 简写）中断函数返回值，不可做其他用途
28	$gp	global pointer. Points to the middle of the 64K block of memory in the static data segment. （Global Pointer 简写）指向 64k(2^16)大小的静态数据块的中间地址（字面上好像就是这个意思，块的中间）
29	$sp	stack pointer Points to last location on the stack. (Stack Pointer 简写）栈指针，指向的是栈顶
30	$s8/$fp	saved value / frame pointer Preserved across procedure calls

		(Saved/Frame Pointer 简写)帧指针
31	$ra	return address 返回地址，目测也是不可做其他用途

Program Structure

程序结构

本质其实就只是：数据声明+普通文本+程序编码（文件后缀为.s，或者.asm）

数据声明在代码段之后（在其之前也没问题，也更符合高级程序设计的习惯）

Data Declarations

数据声明

- 数据段以 **.data** 为开始标志
- 声明变量后，即在主存中分配空间。

Code

代码

- 代码段以 **.text** 为开始标志
- 其实就是各项指令操作
- 程序入口为 **main：**标志（这个都一样啦）
- 程序结束标志（详见下文）

Comments

注释

MIPS 程序的基本模板如下：

```
# Comment giving name of program and description of function
# 说明下程序的目的和作用
# Template.s
#Bare-bones outline of MIPS assembly language program
```

```
            .data       # variable declarations follow this line
                        # 数据变量声明
                        # ...
            .text       # instructions follow this line
                        # 代码段部分
main:                   # indicates start of code (first instru to execute)
                        # 主程序
                        # ...
# End of program, leave a blank line afterwards to make SPIM happy
```

Data Declarations

数据声明

format for declarations:

声明的格式：

```
name:                storage_type       value(s)
变量名：（冒号别少了）      数据类型           变量值
```

- create storage for variable of specified type with given name and specified value
- value(s) usually gives initial value(s); for storage type .space, gives number of spaces to be allocated
- 通常给变量赋一个初始值；对于**.space**,需要指明需要多少大小空间（bytes)

Note: labels always followed by colon (:)

```
example

var1:         .word    3        # create a single integer variable with
initial value 3
               # 声明一个 word 类型的变量 var1, 并赋值为 3
array1:       .byte    'a','b'  # create a 2-element character array with
elements initialized   #   to  a  and  b
                     # 声明一个存储 2 个字符的数组 array1，并赋值 'a', 'b'
array2:       .space   40       # allocate 40 consecutive bytes, with
storage uninitialized
                                #   could be used as a 40-element character
array, or a
                                #   10-element integer array; a comment
should indicate which!
               # 为变量 array2 分配 40 字节（bytes)未使用的连续空间，当然，
```

```
对于这个变量
                # 到底要存放什么类型的值， 最好事先声明注释下！
```

Load / Store Instructions

加载/保存(也许这里写成读取/写入 可能更易理解一点) 指令集

- 如果要访问内存，不好意思，你只能用 **load** 或者 **store** 指令
- 其他的只能都一律是寄存器操作

load:

```
        lw      register_destination, RAM_source
```

```
#copy word (4 bytes) at source RAM location to destination register.
```

从内存中 复制 RAM_source 的内容到 对应的寄存器中

（lw 中的'w'意为'word',即该数据大小为 4 个字节）

```
        lb      register_destination, RAM_source
```

```
#copy byte at source RAM location to low-order byte of destination register,
# and sign-e.g.tend to higher-order bytes
```

同上， lb 意为 load byte

store word:

```
        sw      register_source, RAM_destination
```

```
#store word in source register into RAM destination
```

#将指定寄存器中的数据 写入 到指定的内存中

```
        sb      register_source, RAM_destination
```

```
#store byte (low-order) in source register into RAM destination
```

load immediate:

```
        li      register_destination, value
```

```
#load immediate value into destination register
```

顾名思义，这里的 li 意为 load immediate

```
example:
        .data
var1:.word    23      # declare storage for var1; initial value is 23
```

```
                                # 先声明一个 word 型的变量 var1 = 3;
     .text
__start:
     lw       $t0, var1         # load contents of RAM location into
register $t0:  $t0 = var1
                                # 令寄存器 $t0 = var1 = 3;
     li       $t1, 5            # $t1 = 5   ("load immediate")
                                # 令寄存器 $t1 = 5;
     sw       $t1, var1         # store contents of register $t1 into RAM:
var1 = $t1
                              # 将 var1 的值修改为$t1 中的值： var1 = $t1 = 5;
     done
```

Indirect and Based Addressing

立即与间接寻址

load address:

直接给地址

```
        la       $t0, var1
```

- copy RAM address of var1 (presumably a label defined in the program) into register $t0

indirect addressing:

地址是寄存器的内容（可以理解为指针）

```
        lw       $t2, ($t0)
```

- load word at RAM address contained in $t0 into $t2

```
        sw       $t2, ($t0)
```

- store word in register $t2 into RAM at address contained in $t0

based or indexed addressing：

+偏移量

```
lw      $t2, 4($t0)
```

- load word at RAM address ($t0+4) into register $t2
- "4" gives offset from address in register $t0

```
sw      $t2, -12($t0)
```

- store word in register $t2 into RAM at address ($t0 - 12)
- negative offsets are fine

Note: based addressing is especially useful for:

不必多说，要用到偏移量的寻址，基本上使用最多的场景无非两种：数组，栈。

- arrays; access elements as offset from base address
- stacks; easy to access elements at offset from stack pointer or frame pointer

example：

```
            .data
array1:     .space  12              #  declare 12 bytes of storage to hold array of 3 integers
                    #  定义一个 12 字节 长度的数组 array1, 容纳 3 个整型
            .text
__start:    la      $t0, array1     #  load base address of array into register $t0
                    #  让 $t0 = 数组首地址
            li      $t1, 5          #  $t1 = 5   ("load immediate")
            sw $t1, ($t0)           #  first array element set to 5; indirect addressing
                    # 对于 数组第一个元素赋值 array[0] = $1 = 5
            li $t1, 13              #   $t1 = 13
            sw $t1, 4($t0)          #  second array element set to 13
                    # 对于 数组第二个元素赋值 array[1] = $1 = 13
                    # (该数组中每个元素地址相距长度就是自身数据类型长度，即 4 字节， 所以对于 array+4 就是 array[1])
            li $t1, -7              #   $t1 = -7
            sw $t1, 8($t0)          #  third array element set to -7
                    # 同上， array+8 = （address[array[0])+4）+ 4 = address(array[1]) + 4 = address(array[2])
            done
```

Arithmetic Instructions

算术指令集

- 最多 3 个操作数
- 再说一遍，在这里，操作数只能是寄存器，绝对不允许出现地址
- 所有指令统一是 32 位 = 4 * 8 bit = 4bytes = 1 word

add $t0,$t1,$t2 # $t0 = $t1 + $t2; add as signed (2's complement) integers

```
        sub     $t2,$t3,$t4       #  $t2 = $t3 Ð $t4
        addi    $t2,$t3, 5        #  $t2 = $t3 + 5;   "add immediate" (no sub
    immediate)
        addu    $t1,$t6,$t7       #  $t1 = $t6 + $t7;   add as unsigned
    integers
        subu    $t1,$t6,$t7       #  $t1 = $t6 + $t7;   subtract as unsigned
    integers

        mult    $t3,$t4           #  multiply 32-bit quantities in $t3 and
    $t4, and store 64-bit
                                  #  result in special registers Lo and Hi:
    (Hi,Lo) = $t3 * $t4
```

运算结果存储在 hi,lo（hi 高位数据， lo 地位数据）

```
        div     $t5,$t6           #  Lo = $t5 / $t6   (integer quotient)
                                  #  Hi = $t5 mod $t6   (remainder)
```

商数存放在 lo, 余数存放在 hi

```
        mfhi    $t0               #  move quantity in special register Hi to
    $t0:   $t0 = Hi
```

不能直接获取 hi 或 lo 中的值， 需要 mfhi, mflo 指令传值给寄存器

```
        mflo    $t1               #  move quantity in special register Lo to
    $t1:   $t1 = Lo
                                  #  used to get at result of product or
    quotient
        move    $t2,$t3           #  $t2 = $t3
```

Control Structures

控制流

Branches

分支（if else 系列）

- comparison for conditional branches is built into instruction

```
            b       target          #  unconditional branch to program
label target
            beq     $t0,$t1,target  #  branch to target if  $t0 = $t1
            blt     $t0,$t1,target  #  branch to target if  $t0 < $t1
            ble     $t0,$t1,target  #  branch to target if  $t0 <= $t1
            bgt     $t0,$t1,target  #  branch to target if  $t0 > $t1
            bge     $t0,$t1,target  #  branch to target if  $t0 >= $t1
            bne     $t0,$t1,target  #  branch to target if  $t0 <> $t1
```

Jumps

跳转（while, for, goto 系列）

```
    j       target      #  unconditional jump to program label target
                        看到就跳， 不用考虑任何条件
    jr      $t3             #  jump to address contained in $t3 ("jump
register")
                        类似相对寻址，跳到该寄存器给出的地址处
```

Subroutine Calls

子程序调用

subroutine call: "jump and link" instruction

```
    jal     sub_label       #  "jump and link"
```

- copy program counter (return address) to register $ra (return address register)
- 将当前的程序计数器保存到 $ra 中
- jump to program statement at sub_label

subroutine return: "jump register" instruction

```
    jr      $ra     #  "jump register"
```

- jump to return address in $ra (stored by jal instruction)
- 通过上面保存在 $ra 中的计数器返回调用前

Note: return address stored in register $ra; if subroutine will call other subroutines, or is recursive, return address should be copied from $ra onto stack to preserve it, since jal always places return address in this register and hence will overwrite previous value

如果说调用的子程序中有调用了其他子程序，如此往复， 则返回地址的标记就用 栈（stack） 来存储，毕竟 $ra 只有一个， （哥哥我分身乏术啊~~）。

System Calls and I/O (SPIM Simulator)

系统调用 与 输入/输出(主要针对 SPIM 模拟器）

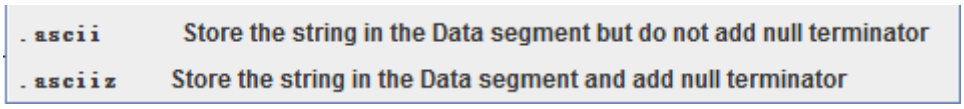
（本人使用的是 Mars 4.4，也通用--！）

- 通过系统调用实现终端的输入输出，以及声明程序结束
- 学会使用 syscall
- 参数所使用的寄存器：$v0， $a0， $a1
- 返回值使用： $v0

下表给出了系统调用中对应功能，代码，参数机返回值

Service	Code in $v0 对应功能的调用码	Arguments 所需参数	Results 返回值
print_int 打印一个整型	$v0 = 1	$a0 = integer to be printed 将要打印的整型赋值给 $a0	
print_float 打印一个浮点	$v0 = 2	$f12 = float to be printed 将要打印的浮点赋值给 $f12	
print_double	$v0	$f12 = double to be printed	

打印双精度	= 3	将要打印的双精度赋值给 $f12	
print_string	$v0 = 4	$a0 = address of string in memory 将要打印的字符串的地址赋值给 $a0	
read_int	$v0 = 5		integer returned in $v0 将读取的整型赋值给 $v0
read_float 读取浮点	$v0 = 6		float returned in $v0 将读取的浮点赋值给 $v0
read_double 读取双精度	$v0 = 7		double returned in $v0 将读取的双精度赋值给 $v0
read_string 读取字符串	$v0 = 8	$a0 = memory address of string input buffer 将读取的字符串地址赋值给 $a0 $a1 = length of string buffer (n) 将读取的字符串长度赋值给 $a1	
sbrk 应该同 C 中的 sbrk() 函数 动态分配内存	$v0 = 9	$a0 = amount 需要分配的空间大小（单位目测是字节 bytes）	address in $v0 将分配好的空间首地址给 $v0
exit 退出	$v0 =10		

- 大概意思是要打印的字符串应该有一个终止符，估计类似 C 中的'\0'，在这里我们只要声明字符串为 **.asciiz** 类型即可。下面给个我用 Mars4.4 的提示：
- 

- .ascii 与 .asciiz 唯一区别就是 后者会在字符串最后自动加上一个终止符， 仅此而已
- The read_int, read_float and read_double services read an entire line of input up to and including the newline character.

- 对于读取整型， 浮点型，双精度的数据操作， 系统会读取一整行，（也就是说以换行符为标志 '\n'）
- The read_string service has the same semantices as the UNIX library routine fgets.
  - It reads up to n-1 characters into a buffer and terminates the string with a null character.
  - If fewer than n-1 characters are in the current line, it reads up to and including the newline and terminates the string with a null character.
  - 这个不多说了，反正就是输入过长就截取，过短就这样，最后都要加一个终止符。
- The sbrk service returns the address to a block of memory containing n additional bytes. This would be used for dynamic memory allocation.
- 上边的表里已经说得很清楚了。
- The exit service stops a program from running.

```
e.g. Print out integer value contained in register $t2
例子：  打印一个存储在寄存器 $2 里的整型

     li       $v0, 1           # load appropriate system call code into
register $v0;
                              声明需要调用的操作代码为 1 （print_int）并赋值
给 $v0
                                        # code for printing integer is 1
     move     $a0, $t2         # move integer to be printed into $a0:  $a0
= $t2
                              将要打印的整型赋值给 $a0
     syscall                  # call operating system to perform operation
e.g.   Read integer value, store in RAM location with label int_value
(presumably declared in data section)

例子：  读取一个数，并且存储到内存中的 int_value 变量中
     li       $v0, 5           # load appropriate system call code into
register $v0;
                              # code for reading integer is 5
                         声明需要调用的操作代码为 5 （read_int）并赋值给 $v0
     syscall                  # call operating system to perform
operation、
                              经过读取操作后， $v0 的值已经变成了 输入的 5
     sw       $v0, int_value   # value read from keyboard returned in
register $v0;
                              # store this in desired location
                       通过写入（store_word)指令 将 $v0 的值（5） 存入 内存中
```

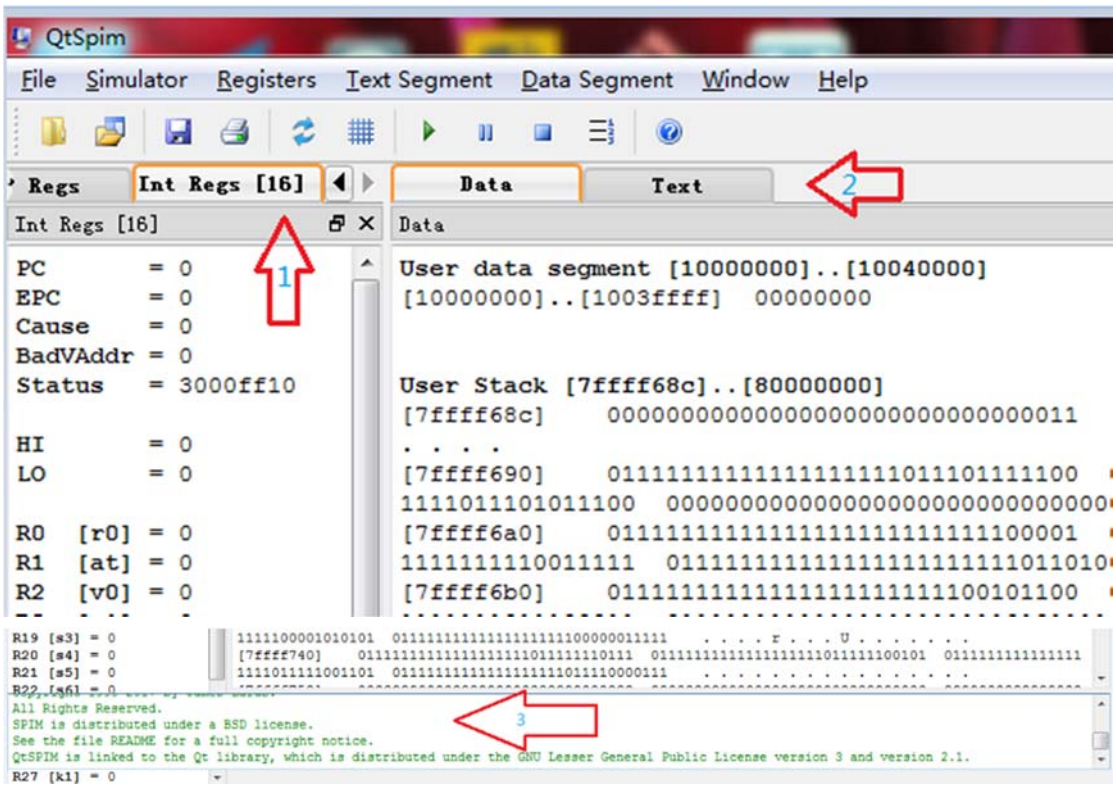
e.g. Print out string (useful for prompts)
栗子： 打印一个字符串(这是完整的，其实上面栗子都可以直接替换 main: 部分，都能直接运行)

```
     .data
string1       .asciiz  "Print this.\n"  # declaration for string variable,
                       # .asciiz directive makes string null terminated
     .text
main:         li       $v0, 4           # load appropriate system call code
into register $v0;
                                        # code for printing string is 4
                              打印字符串， 赋值对应的操作代码 $v0 = 4
     la       $a0, string1             # load address of string to be
printed into $a0
                                 将要打印的字符串地址赋值  $a0 = address(string1)
     syscall           # call operating system to perform print operation
```

*e.g. To indicate end of program, use **exit** system call; thus last lines of program should be:*
执行到这里， 程序结束

```
     li       $v0, 10                # system call code for exit = 10
     syscall                      # call operating sys
```

QtSpim 教程



1 号箭头：Regs 和 Int Regs [16]选项卡：

显示 整型（16 位）、浮点型寄存器；

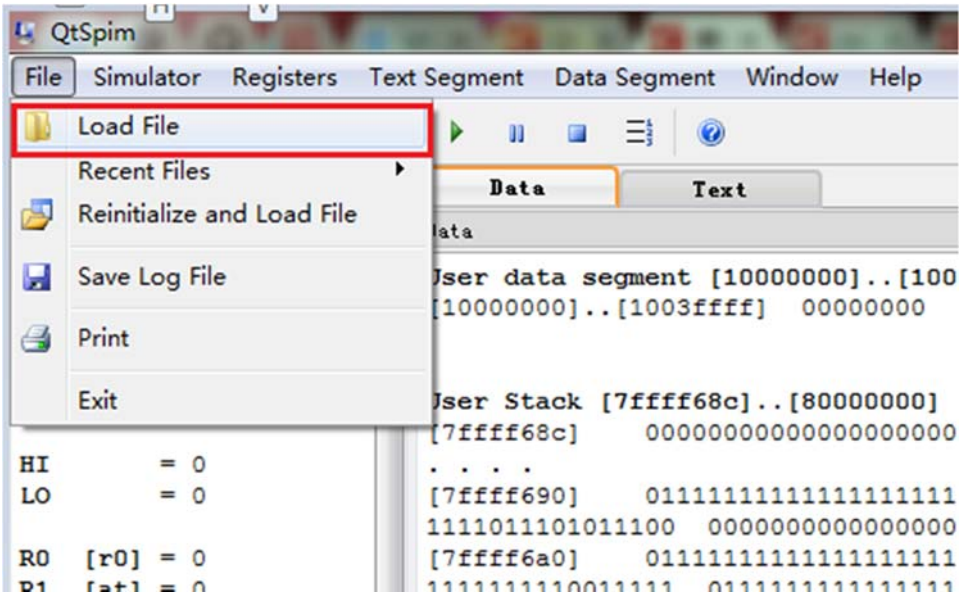
2 号箭头： Data Text 选项卡 ：

text segment ： 显示指令

data segment： 显示数据

3 号箭头：Qtspim 的运行信息

一、 程序



（1）加载汇编文件

file- load file 可加载以 xxx.s 后缀的汇编文件；

汇编文件的编写，可以使用普通的记事本

（2）Reinitialize and Load File

初始化并加载最后一次的文件；用于 debug

（3）运行程序：

1.

2．simulator -run

可在 console 看到结果，window 将 console 打上勾；

二、 调试

（1）单步运行

1 simulator – single step

2 

（2）设置断点

指令上右键 setbreakpoint

点击 run 会运行到断点；

三、运行、暂停、停止

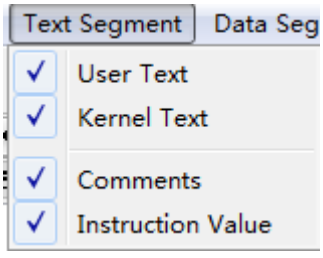
（1）pause 再点击 run 会继续运行 

（2）停止 再点击 run 会从头开始运行  ；

四、 显示选项

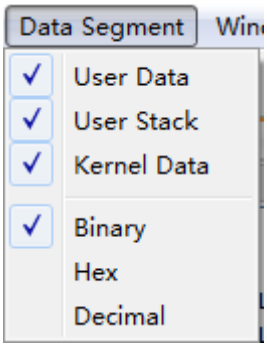
（1）registers 决定以何种进制显示寄存器中信息

（2）text segment 对 text 标签页进行修改



分别为 用户信息、内核信息、注释、指令值； 打勾代表在窗口中显示该项信息

（3）data segment 对 data 标签页进行修改



设定在 data 标签页 显示哪些内容、并以何种进制显示;

五、 模拟器设定

Simulator –settings

出现两个标签页

（1）Qtspim

可设置字体、背景颜色、显示最近文件的个数等

（2）MIPS

1 bare machine 是 qtspim 模拟 精简的 spim 处理器

2 accept pseudo instruction

是否接受伪代码

3 enable delayed branches:

启用分支延迟;

4 enable delayed loads:

启用加载延迟

5 enable mapped IO

启用映射 I/O

（3）快捷设置：



Simple machine ： 对应普通人的设置；

Bare machine ： 对应真实的 MIPS 处理器

六、实例

（1）编写汇编程序

```
##
## print “hello world”
##  programed by: stevie zou
##
##########################################
#  text segment      #
##########################################

   .text
   .globl main
main:

    la $a0,str    #$a0 储存要打印字符的地址
    li $v0, 4     #为syscall 读取整数值
    syscall       #system call service 4  打印字符串

    li $v0,10
    syscall       #system call service 10 退出

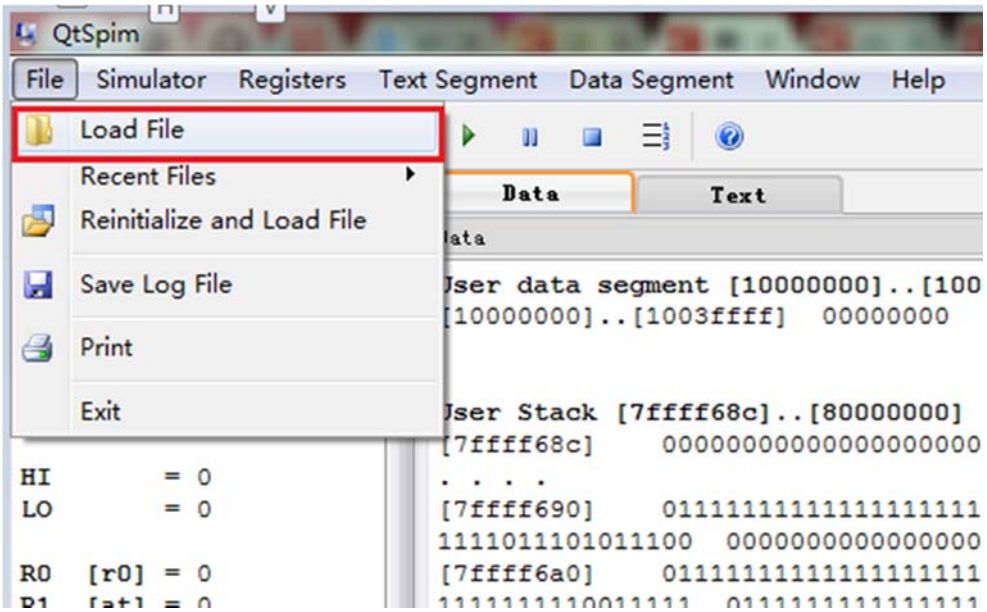
##########################################
#   data segment      #
##########################################

    .data
str: .asciiz "hello world/n"
##
#############end of file
```

可用记事本编写，并保存为 hello_world.s

（2）加载并编译运行

1.加载 hello_world.s 文件

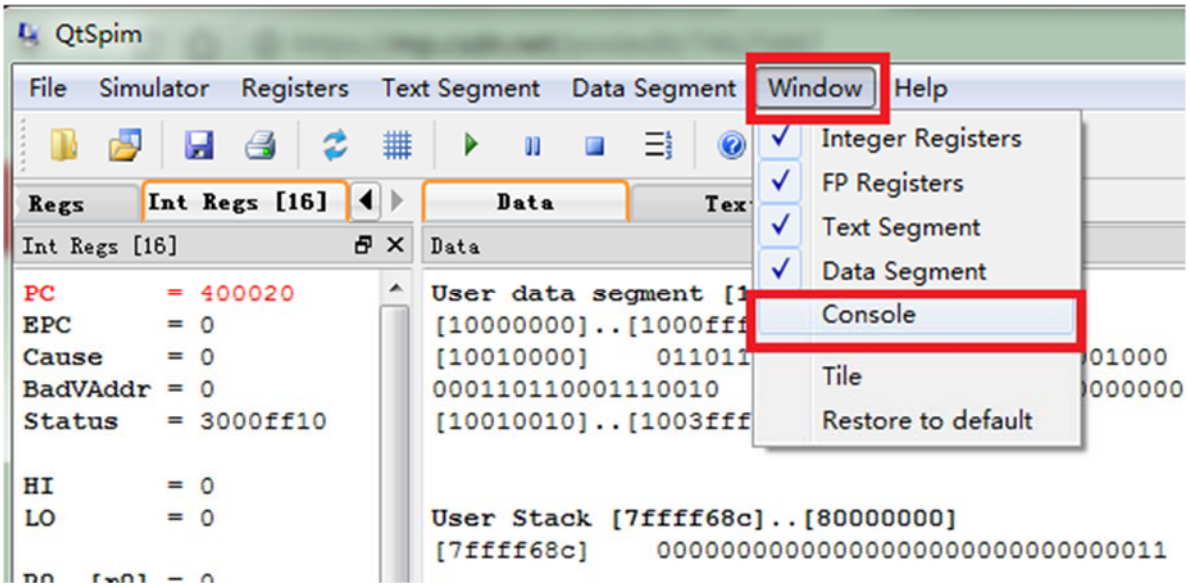


2.运行 hello_world.s 文件

点击图标  以运行、暂停、停止 或 点击 simulator — run 以运行；

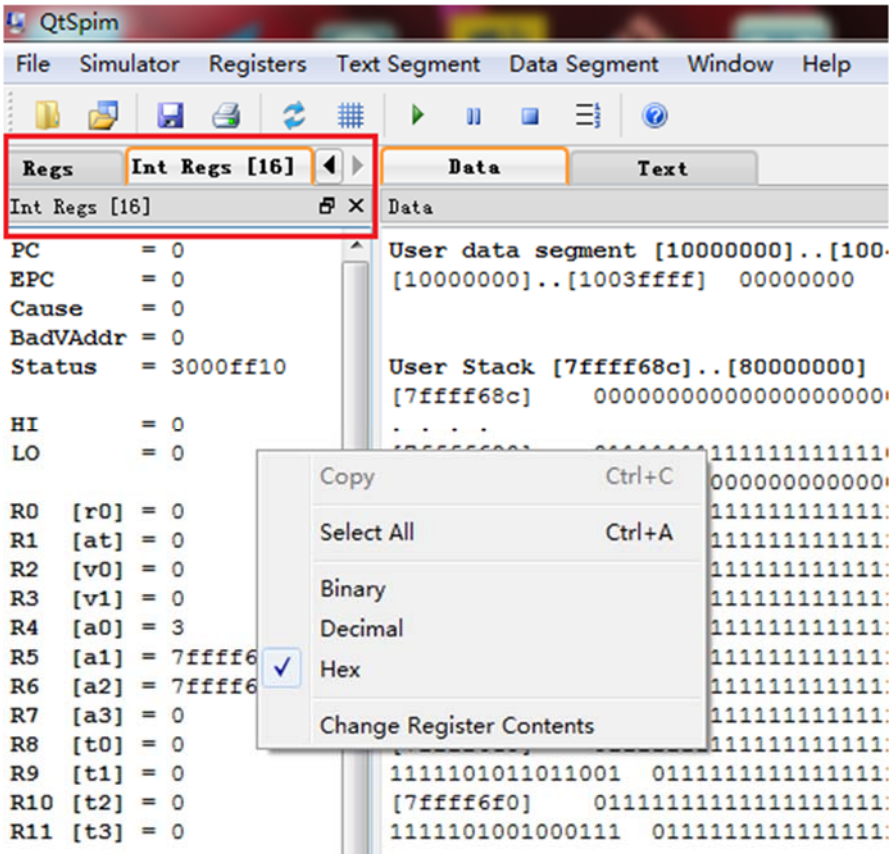
3.观察输出结果

若没有出现 console 窗口，可点击 window — 勾选 console ，以显示结果



4. 观察寄存器值

在左侧 Int Regs[16]窗口可观察寄存器的值



更改寄存器值以何种进制显示：

空白处右键，可分别选择 Binary（二进制），Decimal（十进制），Hex （十六进制）

（3）调试程序

1. 每点击 Simulator— Single Step 一次，程序便从上一次执行处向下单步执行

2. F10

3 点击图标 

注：qtspim 不支持设置断点

每单步执行一次，PC 中的值便会增加，增加的值取决于指令的长度。

完成三个实验：

1. 打印 Hello；

2. 完成两个数的加法运算并输出。

3. 完成加，减，乘、除四则运算计算器设计。

每人以实验报告的形式将电子版上交：考试前一周学在西电公布上交方式。